

TREND STORE APPLICATION DEPLOYMENT USING TERRAFORM, JENKINS, DOCKER, AWS EKS, PROMETHEUS AND GRAFANA

ABSTRACT

This project demonstrates the complete deployment and monitoring lifecycle of the Trend Store application using modern DevOps practices. Infrastructure provisioning was automated using Terraform, Jenkins was used for Continuous Integration and Continuous Deployment (CI/CD), Docker was used for application containerization, and Amazon Elastic Kubernetes Service (EKS) was used for orchestration and deployment. Monitoring and visualization were implemented using Prometheus and Grafana to observe cluster health and resource utilization. The project successfully integrates Infrastructure as Code, CI/CD automation, containerization, cloud deployment, and monitoring into a unified workflow.

1. INTRODUCTION

Modern applications require automated deployment and continuous monitoring to ensure reliability and scalability. DevOps practices enable organizations to automate software delivery while maintaining application performance and availability. The objective of this project is to deploy the Trend Store application on AWS EKS using Docker containers and automate the deployment process through Jenkins. Monitoring is implemented using Prometheus and Grafana to provide real-time insights into system performance.

2. OBJECTIVES

The objectives of this project are:

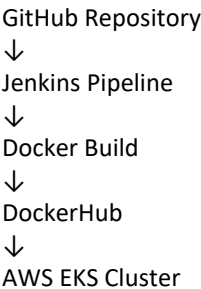
- Provision infrastructure using Terraform.
- Install and configure Jenkins.
- Containerize the application using Docker.
- Push Docker images to DockerHub.
- Deploy the application on AWS EKS.
- Automate deployment using Jenkins Pipeline.
- Implement monitoring using Prometheus and Grafana.
- Visualize cluster metrics and resource utilization.

3. TOOLS AND TECHNOLOGIES USED

Tool	Purpose
AWS EC2	Virtual Machine
Terraform	Infrastructure Provisioning
Jenkins	CI/CD Automation
Docker	Containerization
DockerHub	Image Repository
GitHub	Source Code Management
Kubernetes (EKS)	Container Orchestration
Prometheus	Monitoring
Grafana	Visualization Dashboard

4. SYSTEM ARCHITECTURE

The workflow implemented in this project is:



↓
Application Deployment
↓
Prometheus Monitoring
↓
Grafana Dashboard

5. INFRASTRUCTURE PROVISIONING USING TERRAFORM

Terraform was used to automate the creation of cloud resources required for the project.

Terraform configuration files:

- main.tf
- variables.tf
- outputs.tf

Terraform commands executed:

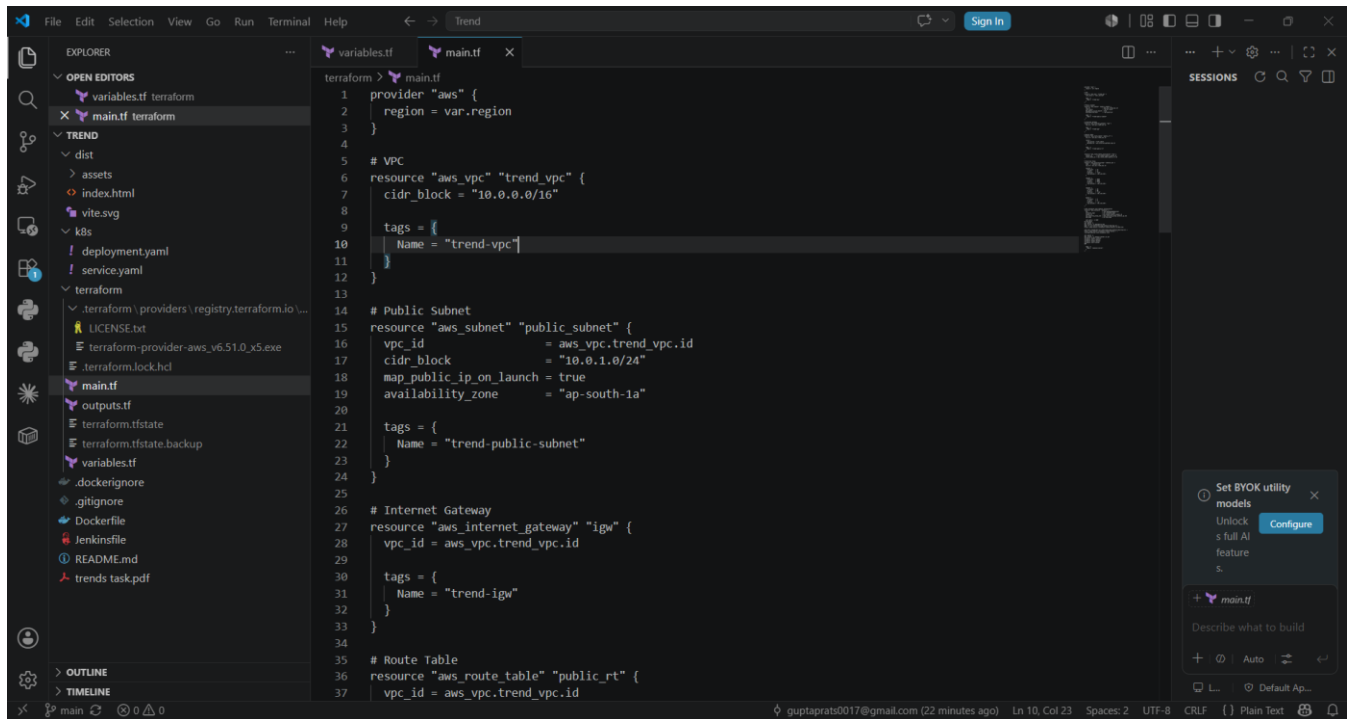
terraform init

terraform plan

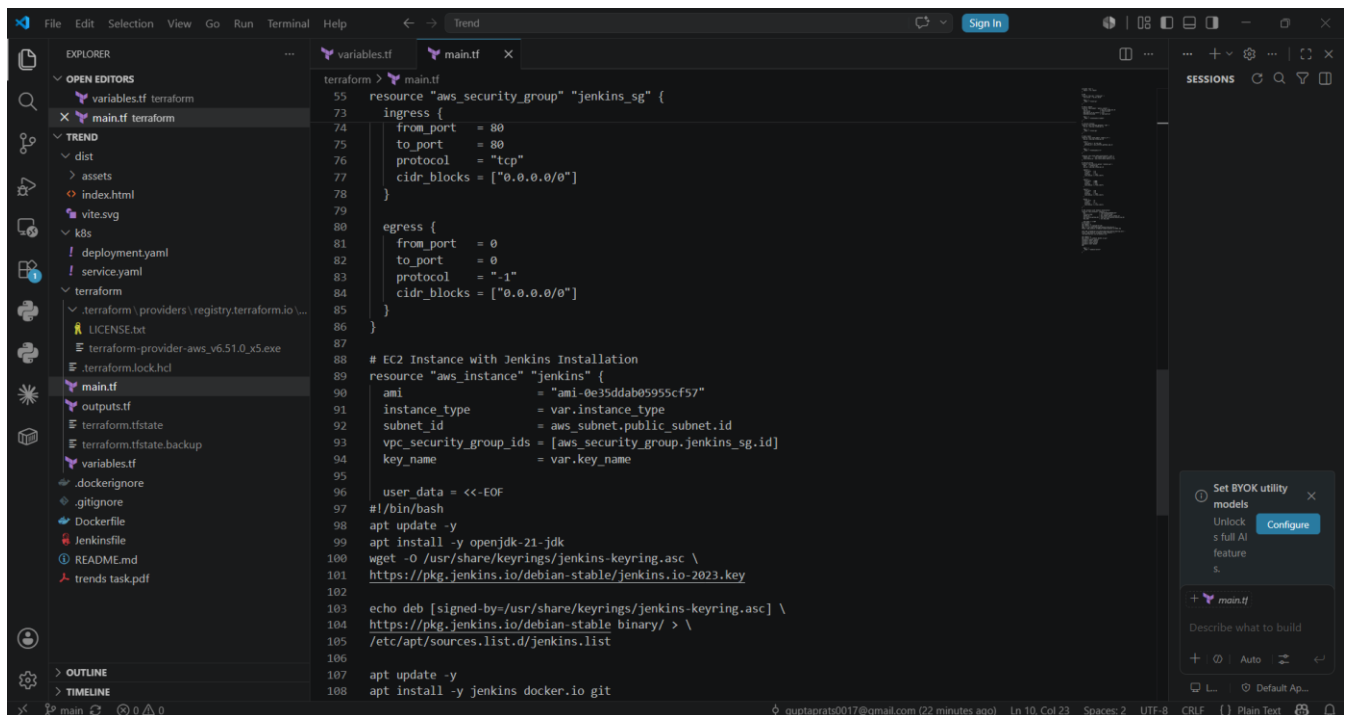
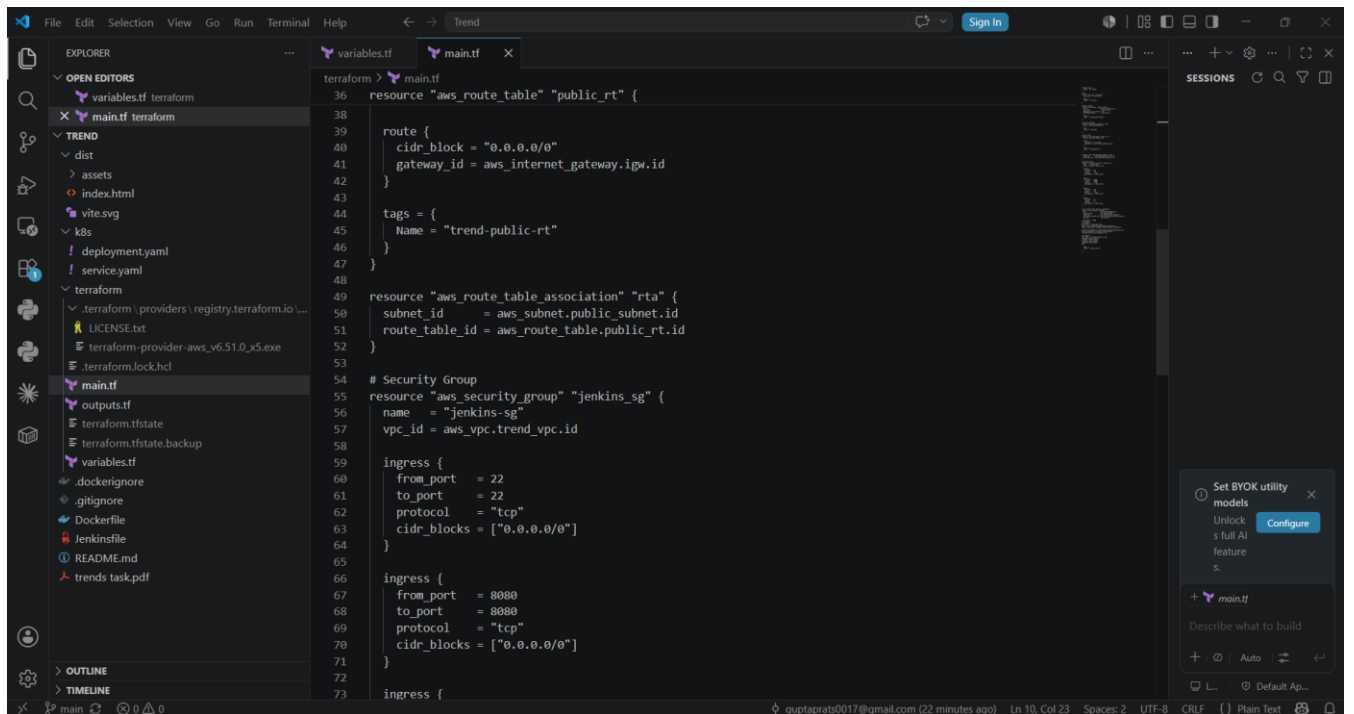
terraform apply

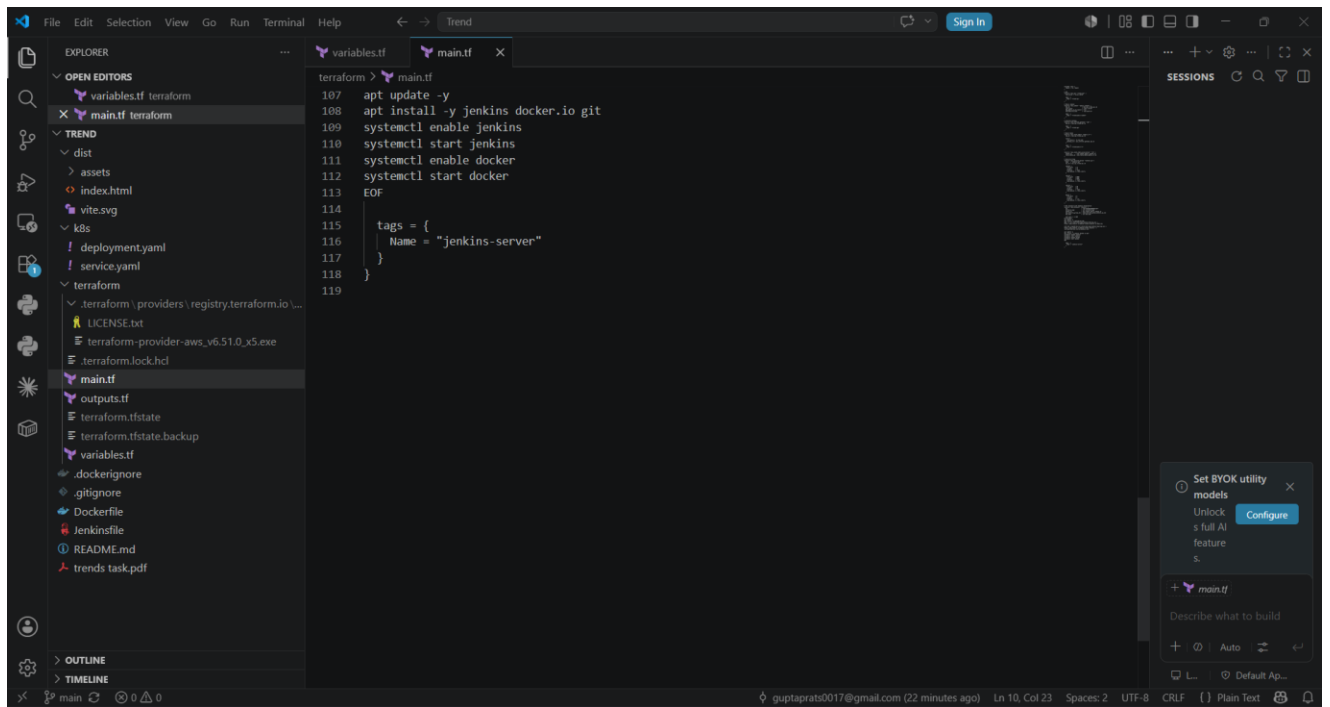
These commands provisioned the EC2 instance used for Jenkins setup.

Main.tf

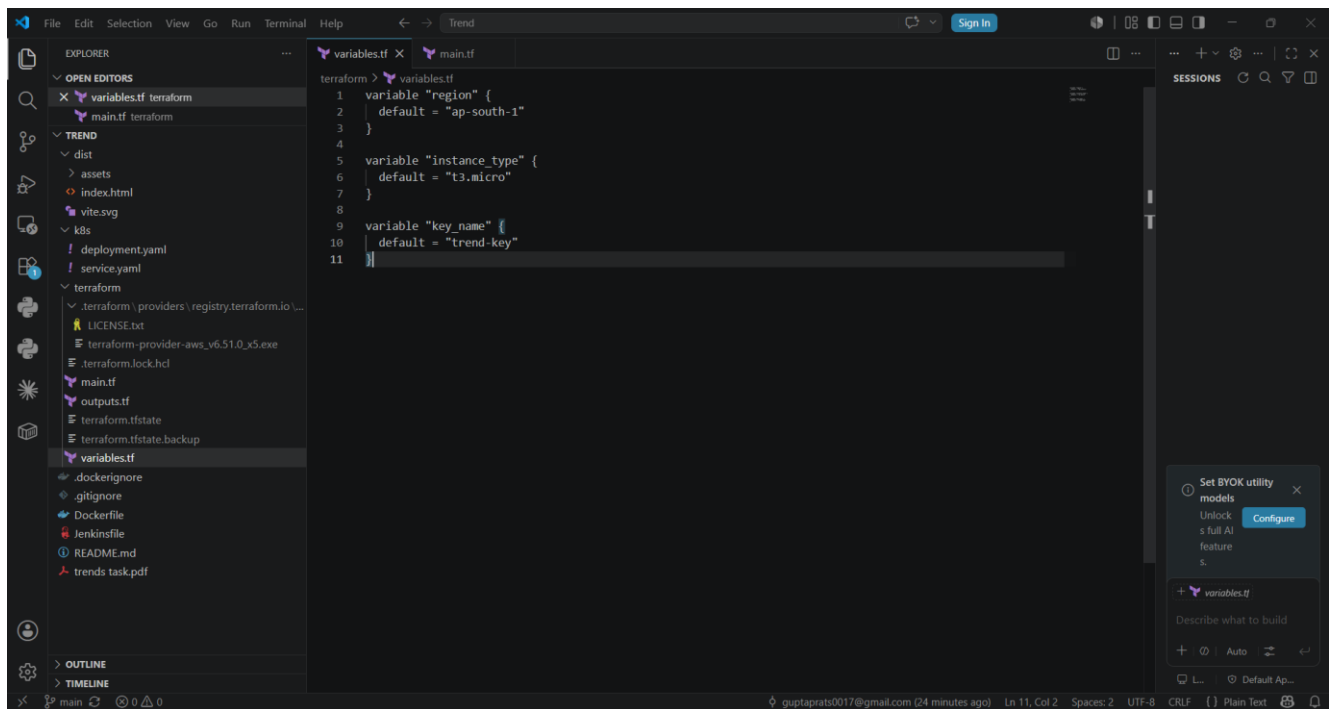


```
1 provider "aws" {
2   region = var.region
3 }
4
5 # VPC
6 resource "aws_vpc" "trend_vpc" {
7   cidr_block = "10.0.0.0/16"
8
9   tags = {
10     Name = "trend-vpc"
11   }
12 }
13
14 # Public Subnet
15 resource "aws_subnet" "public_subnet" {
16   vpc_id            = aws_vpc.trend_vpc.id
17   cidr_block        = "10.0.1.0/24"
18   map_public_ip_on_launch = true
19   availability_zone  = "ap-south-1a"
20
21   tags = {
22     Name = "trend-public-subnet"
23   }
24 }
25
26 # Internet Gateway
27 resource "aws_internet_gateway" "igw" {
28   vpc_id = aws_vpc.trend_vpc.id
29
30   tags = {
31     Name = "trend-igw"
32   }
33 }
34
35 # Route Table
36 resource "aws_route_table" "public_rt" {
37   vpc_id = aws_vpc.trend_vpc.id
```

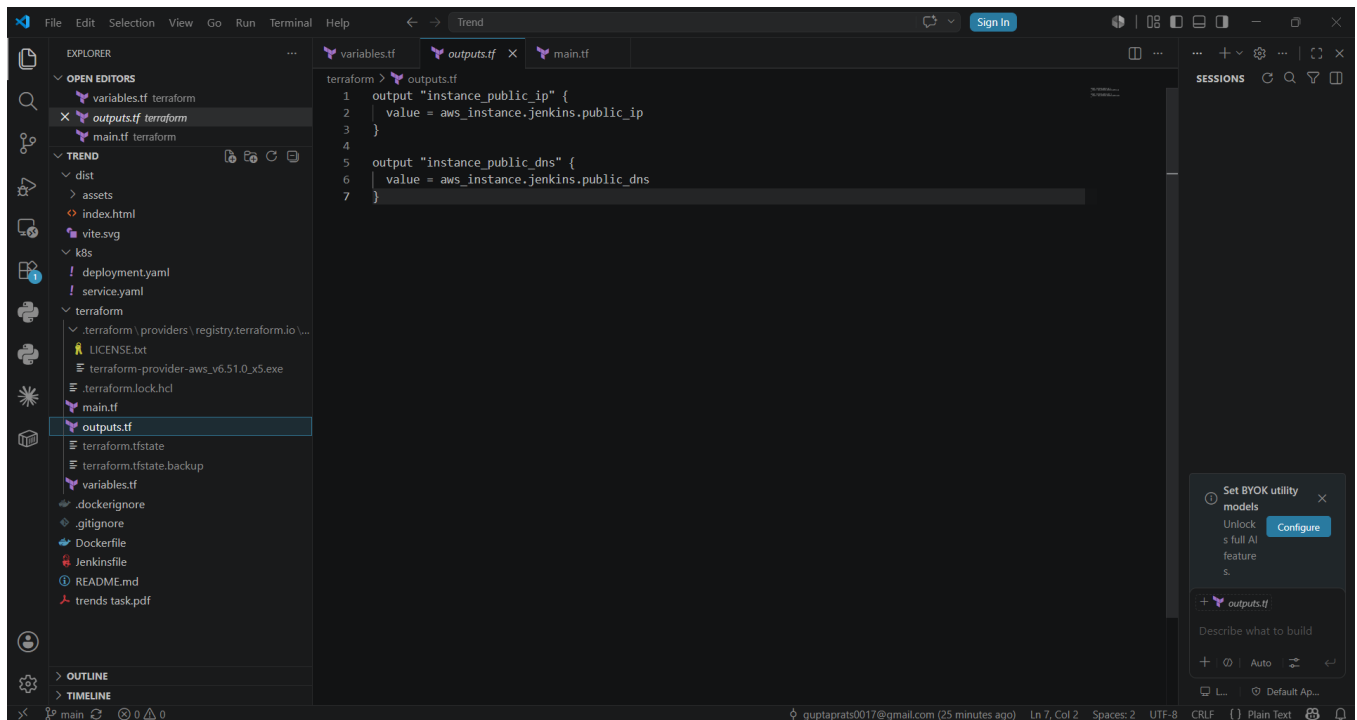




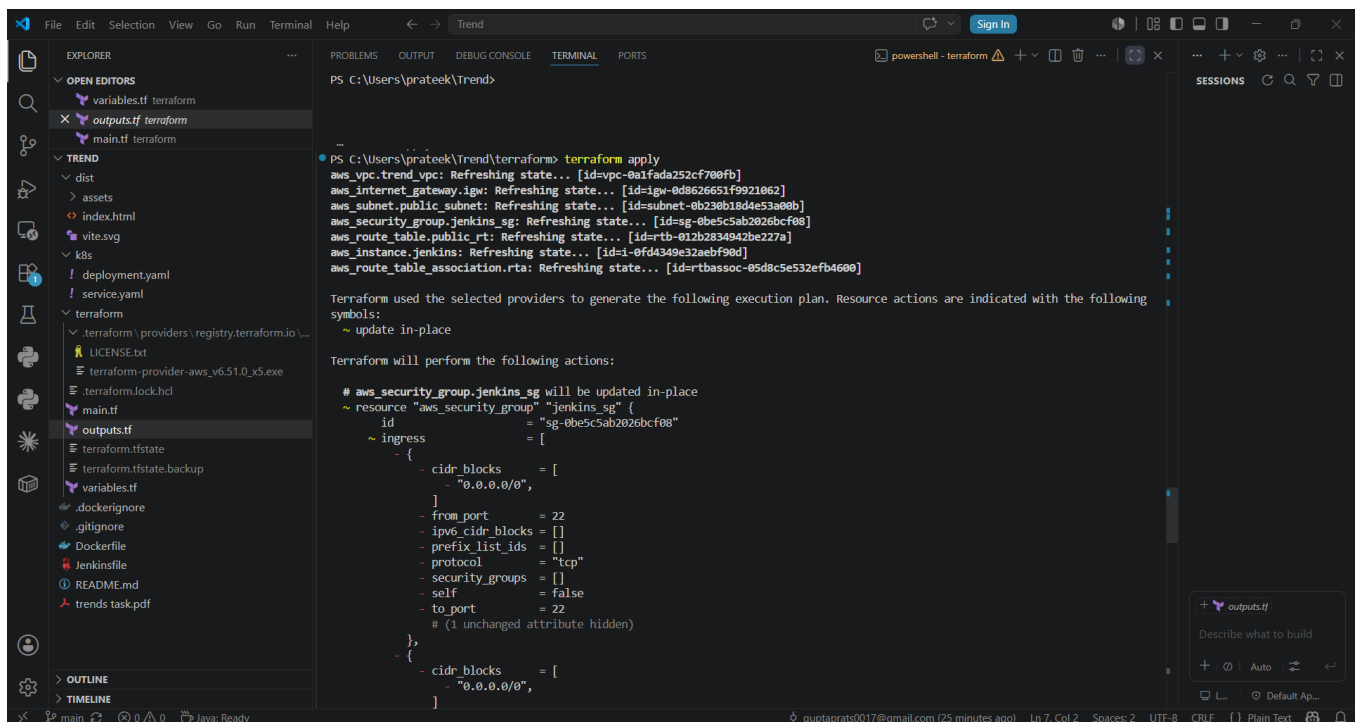
Variables.tf

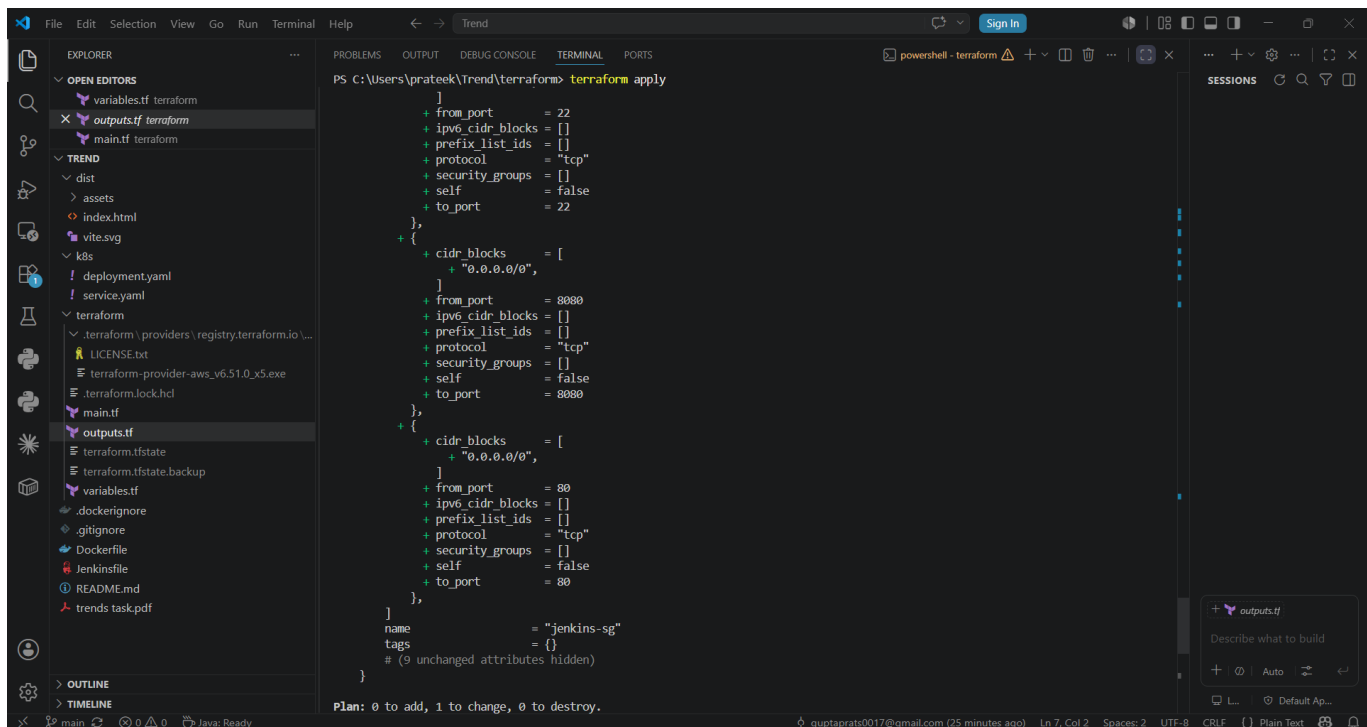
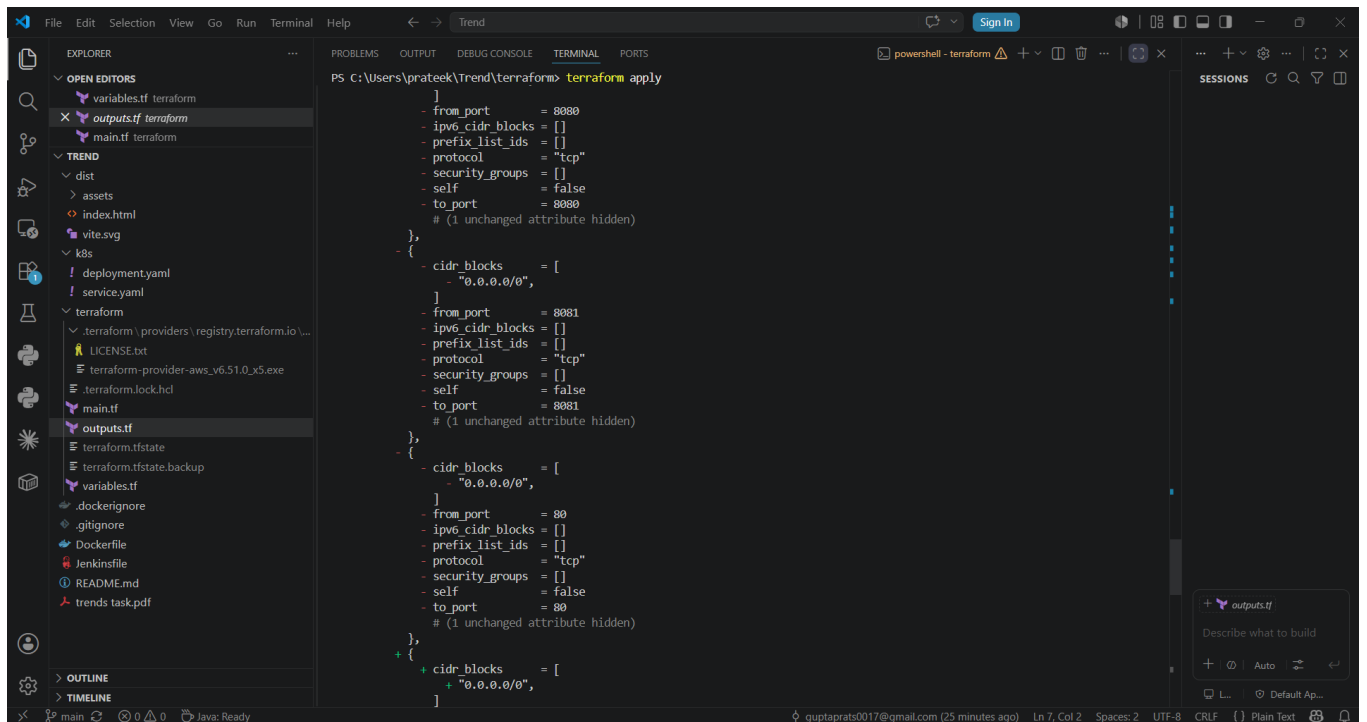


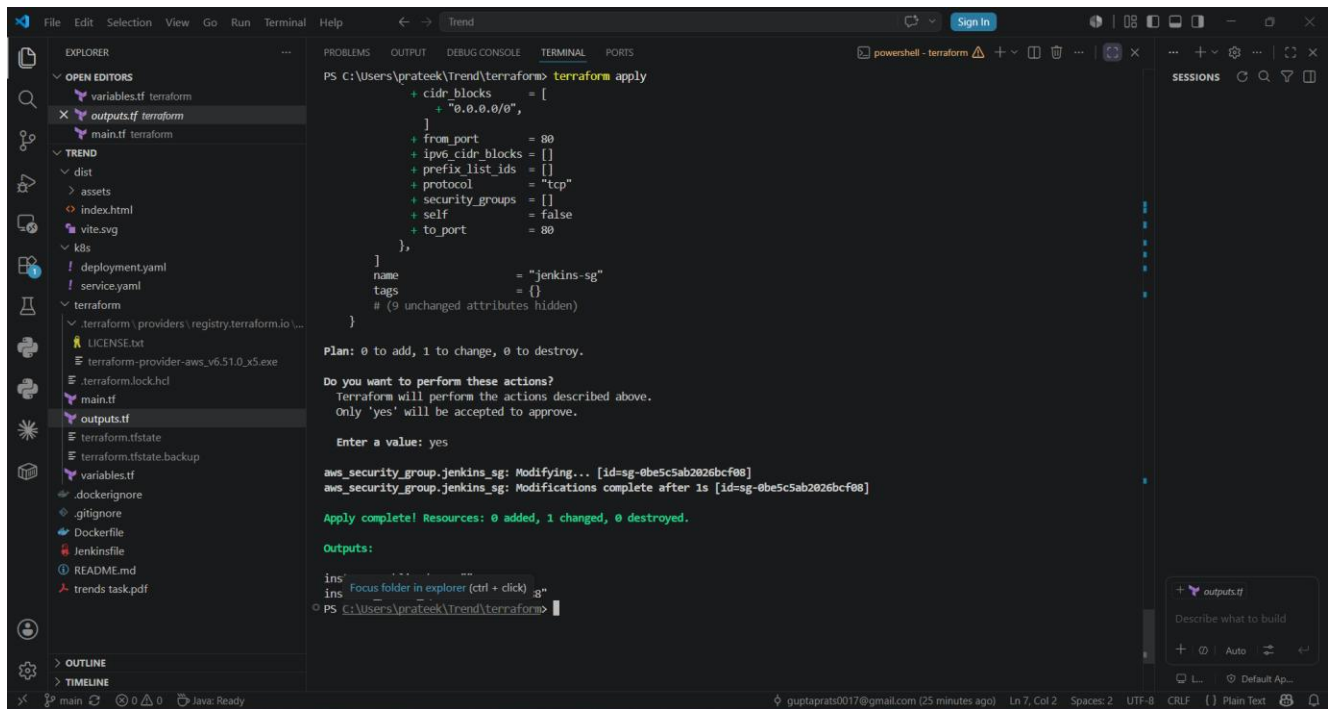
Outputs.tf



Apply output







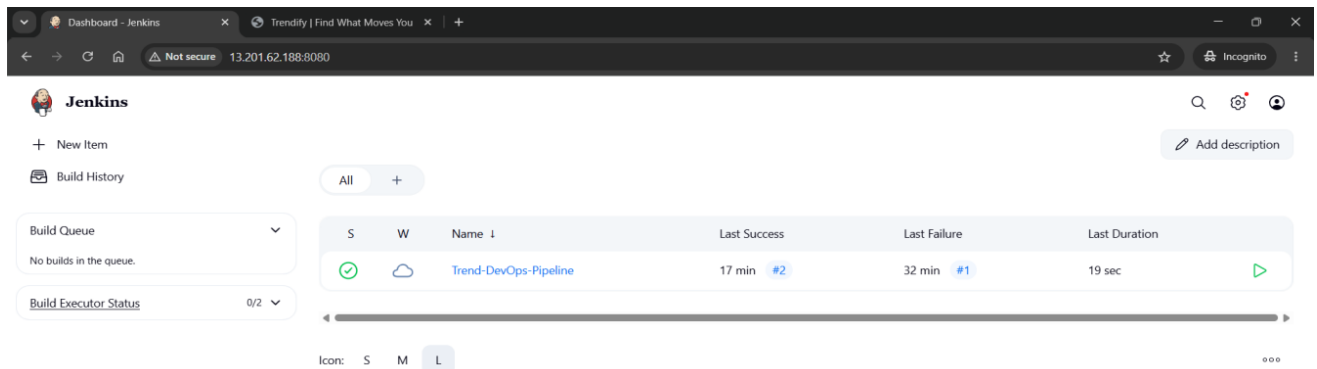
6. JENKINS INSTALLATION AND CONFIGURATION

Jenkins was installed on the EC2 instance and configured for CI/CD automation.

The following plugins were used:

- Docker Plugin
- Git Plugin
- Pipeline Plugin
- Kubernetes CLI Plugin

Jenkins was connected to GitHub for source code retrieval.



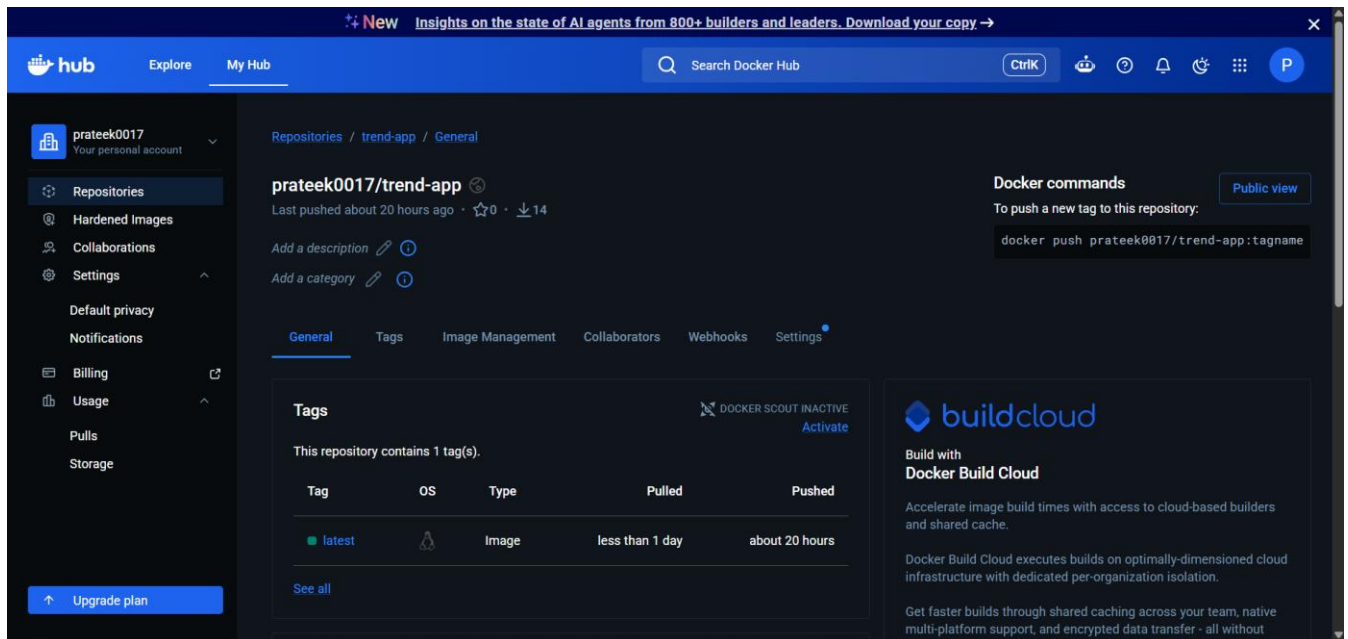
7. APPLICATION CONTAINERIZATION USING DOCKER

The Trend Store application was containerized using Docker.

Docker commands used:

docker build -t prateek0017/trend-app:latest .

docker push prateek0017/trend-app:latest
The image was successfully uploaded to DockerHub.



8. KUBERNETES DEPLOYMENT ON AWS EKS

Amazon Elastic Kubernetes Service (EKS) was used to deploy the application.

The deployment was performed using:

deployment.yaml

service.yaml

Application replicas were managed by Kubernetes deployment.

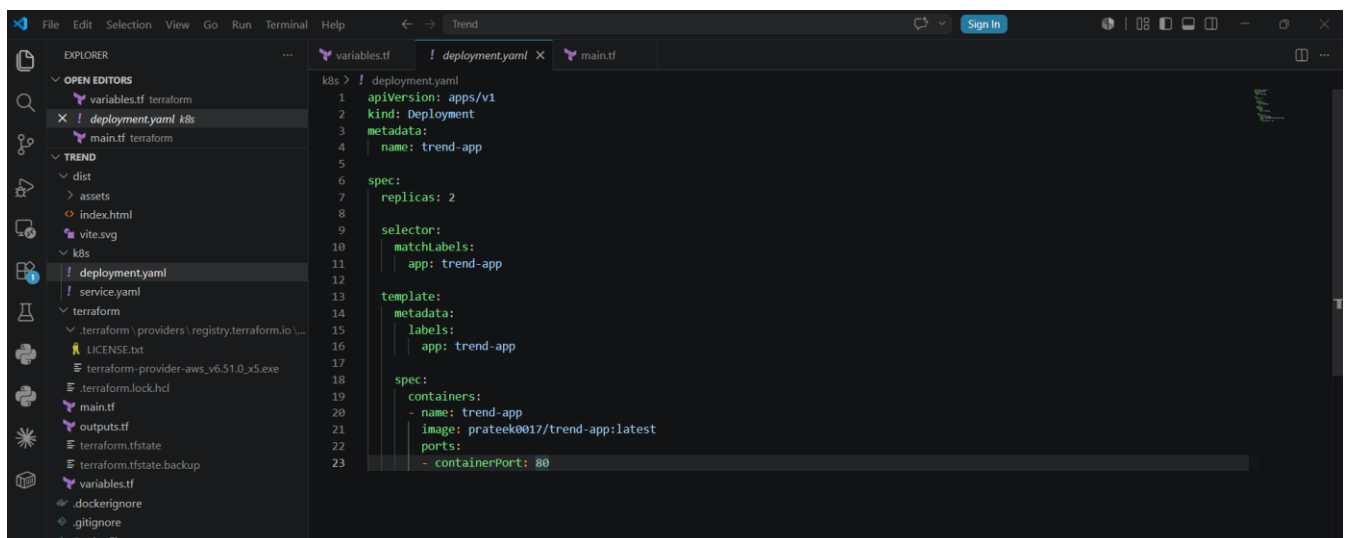
LoadBalancer service exposed the application externally.

Commands executed:

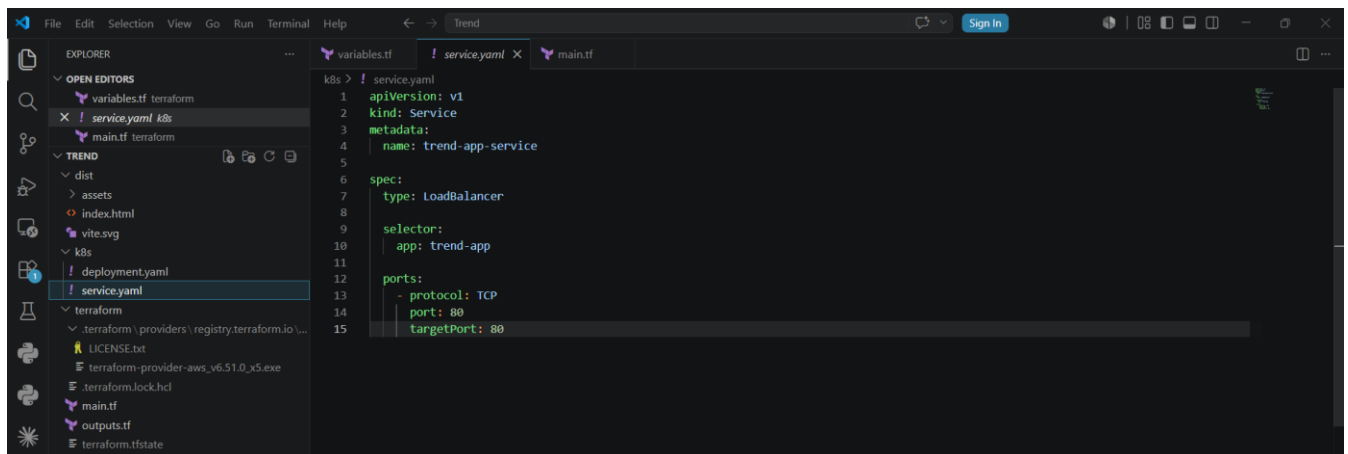
```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

deployment.yaml



service.yaml



kubectl get nodes

```
PS C:\Users\prateek> kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-172-31-19-229.ap-south-1.compute.internal	Ready	<none>	111s	v1.35.5-eks-0de9cde
ip-172-31-44-133.ap-south-1.compute.internal	Ready	<none>	21h	v1.35.5-eks-0de9cde
ip-172-31-6-76.ap-south-1.compute.internal	Ready	<none>	21h	v1.35.5-eks-0de9cde

kubectl get pods

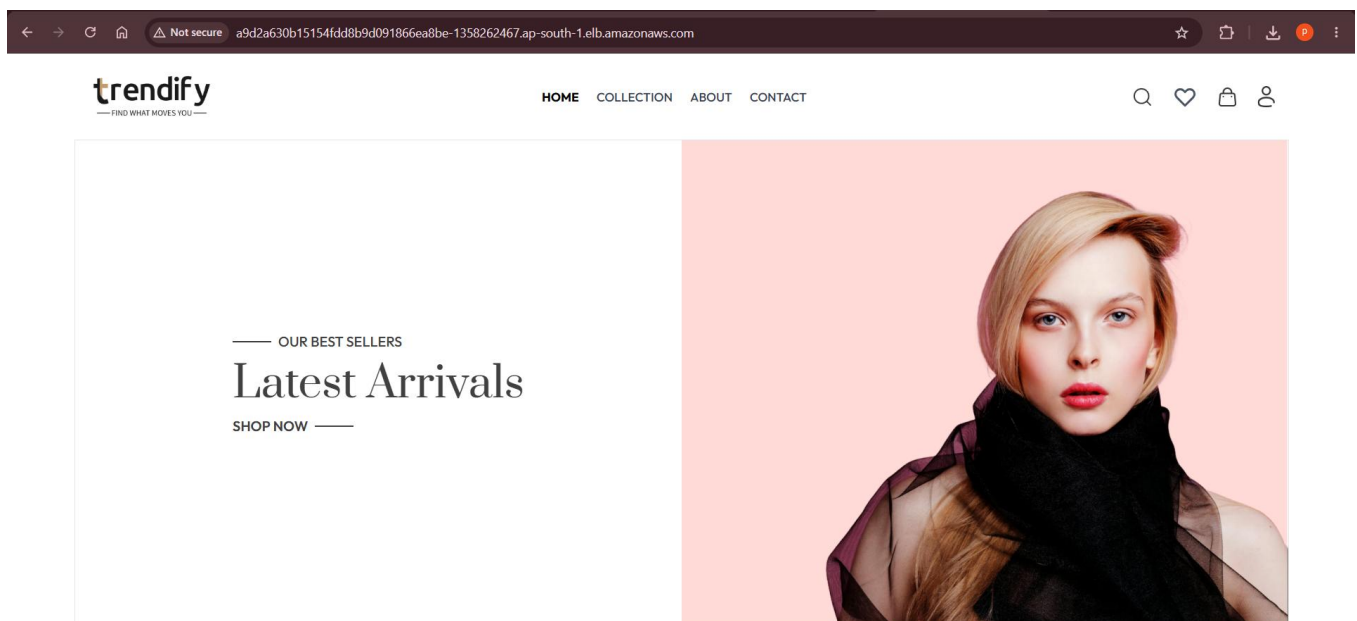
```
PS C:\Users\prateek> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0	2/2	Running	0	18h
brain-app-5b96bb8ff6-jcgqd	1/1	Running	0	22h
brain-app-5b96bb8ff6-zdvqz	1/1	Running	0	22h
brain-tasks-app-6dcbd65485-56cgt	1/1	Running	0	22h
brain-tasks-app-6dcbd65485-dbztz	1/1	Running	0	22h
monitoring-grafana-7cf89c6877-v6bsl	3/3	Running	0	18h
monitoring-kube-prometheus-admission-patch-5n824	0/1	ContainerCreating	0	18h
monitoring-kube-prometheus-operator-688d9c4974-2h2wg	1/1	Running	0	18h
monitoring-kube-state-metrics-6b8f7fb688-4cqfm	1/1	Running	0	18h
monitoring-prometheus-node-exporter-8spsk	1/1	Running	0	7m50s
monitoring-prometheus-node-exporter-94c2d	0/1	Pending	0	18h
monitoring-prometheus-node-exporter-vc89k	1/1	Running	0	18h
nginx-deployment-59f86b59ff-b6hkn	1/1	Running	0	22h
nginx-deployment-59f86b59ff-nz4dg	1/1	Running	0	22h
prometheus-monitoring-kube-prometheus-prometheus-0	2/2	Running	0	18h
trend-app-574fb54f95-czs5z	1/1	Running	0	19h
trend-app-574fb54f95-gq6ht	1/1	Running	0	19h

kubectl get svc

```
PS C:\Users\prateek> kubectl get svc
NAME                                PORT(S)                                AGE      TYPE           CLUSTER-IP      EXTERNAL-IP
alertmanager-operated              9093/TCP,9094/TCP,9094/UDP            19h      ClusterIP      None             <none>
brain-app-service                  80:30587/TCP                          41h      LoadBalancer  10.100.17.247    a59651fbc4cfe4cd2b6f25260cdf82b9-587373243.ap-south-1.elb.amazonaws.co
brain-tasks-service                80:32611/TCP                          24h      LoadBalancer  10.100.133.110   af09f1be503794c82859c92d561276b9-2029089949.ap-south-1.elb.amazonaws.c
grafana-service                    80:32244/TCP                          19h      NodePort       10.100.34.80     <none>
kubernetes                         443/TCP                               5d20h    ClusterIP      10.100.0.1       <none>
monitoring-grafana                 80:30257/TCP                          19h      LoadBalancer  10.100.253.119   a4d0acc5f96b044d3ad3bb69759dbcc7-442839376.ap-south-1.elb.amazonaws.co
monitoring-kube-prometheus-alertmanager 9093/TCP,8080/TCP                    19h      ClusterIP      10.100.129.5     <none>
monitoring-kube-prometheus-operator 443/TCP                              19h      ClusterIP      10.100.86.75     <none>
monitoring-kube-prometheus-prometheus 9090/TCP,8080/TCP                    19h      ClusterIP      10.100.5.164     <none>
monitoring-kube-state-metrics      8080/TCP                              19h      ClusterIP      10.100.66.76     <none>
monitoring-prometheus-node-exporter 9100/TCP                              19h      ClusterIP      10.100.232.165   <none>
nginx-service                      80:31518/TCP                          4d22h    LoadBalancer  10.100.113.107   af38df21daa4c4aaba99773bf5085480-1110248336.ap-south-1.elb.amazonaws.c
prometheus-operated                9090/TCP                              19h      ClusterIP      None             <none>
trend-app-service                  80:30080/TCP                          20h      LoadBalancer  10.100.105.59    a9d2a630b15154fdd8b9d091866ea8be-1358262467.ap-south-1.elb.amazonaws.c
```

LoadBalancer URL



9. CI/CD PIPELINE IMPLEMENTATION

Jenkins pipeline was created to automate the deployment process.

Pipeline stages:

Stage 1: Build Docker Image

Stage 2: Push Image to DockerHub

Stage 3: Deploy Application to EKS

Jenkinsfile used:

```
pipeline {
  agent any
  stages {

    stage('Build Docker Image') {
      steps {
```

```

        sh 'docker build -t prateek0017/trend-app:latest .'
    }
}

stage('Push to DockerHub') {
    steps {
        sh 'docker push prateek0017/trend-app:latest'
    }
}

stage('Deploy to EKS') {
    steps {
        sh 'kubectl apply -f k8s/deployment.yaml'
        sh 'kubectl apply -f k8s/service.yaml'
    }
}
}
}

```

Jenkins / Trend-DevOps-Pipeline

Stage View

	Declarative: Checkout SCM	Build Docker Image	Run Container
Average stage times: (full run time: ~19s)	1s	11s	1s
#2 Jun 23 09:39 No Changes	1s	11s	1s
#1 Jun 23 09:25 No Changes			

Permalinks

- Last build (#2), 18 min ago
- Last stable build (#2), 18 min ago
- Last successful build (#2), 18 min ago
- Last failed build (#1), 32 min ago
- Last unsuccessful build (#1), 32 min ago
- Last completed build (#2), 18 min ago

Builds

Filter

Today

- #2 4:09 am
- #1 3:55 am

10. MONITORING USING PROMETHEUS

Prometheus was deployed using kube-prometheus-stack.

Prometheus collects metrics from:

- Kubernetes Cluster
- Node Exporter
- Pods
- Nodes
- Containers

Prometheus pods were successfully deployed and configured.

Prometheus Pods Running

```
PS C:\Users\prateek> kubectl get pods | findstr prometheus
alertmanager-monitoring-kube-prometheus-alertmanager-0    2/2    Running    0    19h
monitoring-kube-prometheus-admission-patch-5n824          0/1    ContainerCreating    0    19h
monitoring-kube-prometheus-operator-688d9c4974-2h2wg      1/1    Running    0    19h
monitoring-prometheus-node-exporter-8spsk                 1/1    Running    0    74m
monitoring-prometheus-node-exporter-94c2d                 0/1    Pending    0    19h
monitoring-prometheus-node-exporter-vc89k                 1/1    Running    0    19h
prometheus-monitoring-kube-prometheus-prometheus-0        2/2    Running    0    19h
```

Prometheus Service

```
PS C:\Users\prateek> kubectl get svc | findstr prometheus
monitoring-kube-prometheus-alertmanager    ClusterIP    10.100.129.5    <none>
9093/TCP,8080/TCP    20h
monitoring-kube-prometheus-operator        ClusterIP    10.100.86.75    <none>
443/TCP    20h
monitoring-kube-prometheus-prometheus      ClusterIP    10.100.5.164    <none>
9090/TCP,8080/TCP    20h
monitoring-prometheus-node-exporter        ClusterIP    10.100.232.165  <none>
9100/TCP    20h
prometheus-operated                        ClusterIP    None            <none>
9090/TCP    20h
```

11. VISUALIZATION USING GRAFANA

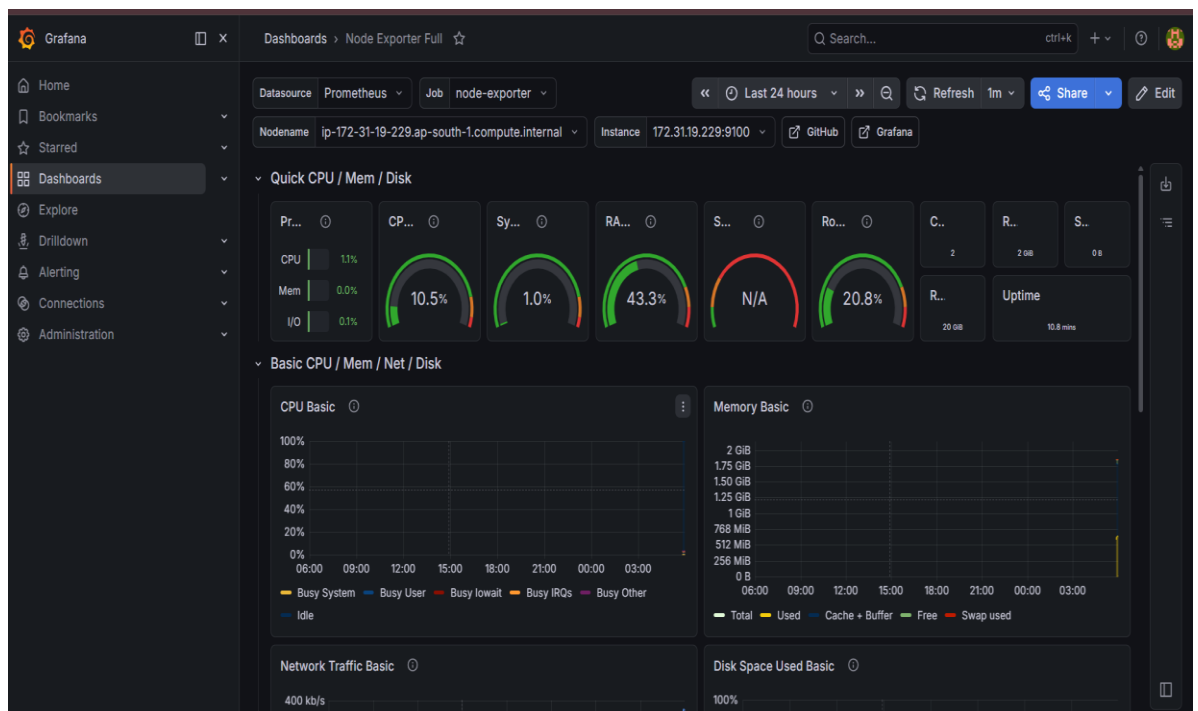
Grafana was integrated with Prometheus as the datasource.

Several dashboards were imported for visualization.

Node Exporter Full Dashboard

The dashboard displays:

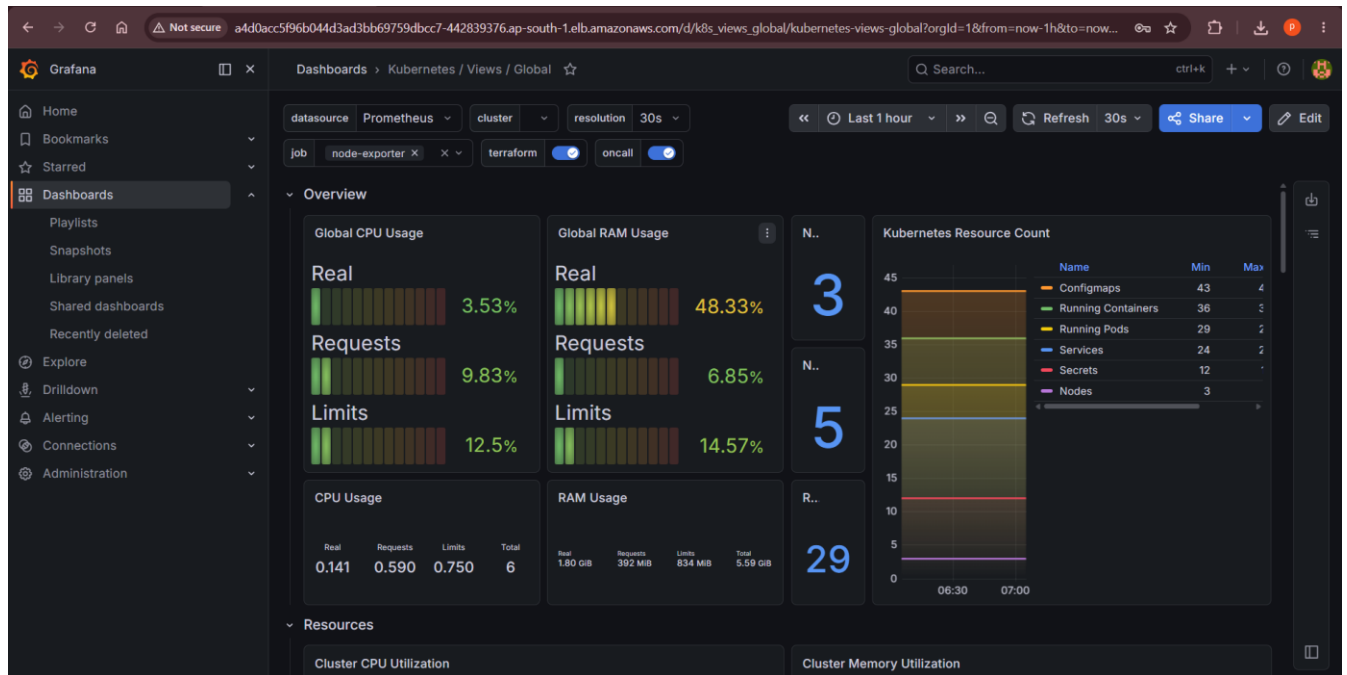
- CPU Utilization
- Memory Utilization
- Disk Usage
- Network Traffic
- System Uptime



Kubernetes Global View Dashboard

The dashboard provides:

- Cluster CPU Usage
- Cluster RAM Usage
- Number of Nodes
- Running Pods
- Running Containers
- Kubernetes Resource Count



12. RESULTS

The Trend Store application was successfully deployed and monitored.

The following objectives were achieved:

- ✓ Infrastructure Provisioning using Terraform
- ✓ Jenkins Installation and Configuration
- ✓ Docker Image Creation
- ✓ DockerHub Integration
- ✓ Kubernetes Deployment on AWS EKS
- ✓ CI/CD Pipeline Automation
- ✓ Prometheus Monitoring
- ✓ Grafana Dashboard Visualization

13. CONCLUSION

The Trend Store application was successfully deployed using a complete DevOps workflow. Infrastructure provisioning was automated using Terraform, while Jenkins handled Continuous Integration and Continuous Deployment. Docker enabled application containerization, and Kubernetes ensured scalable deployment through AWS EKS. Prometheus and Grafana provided comprehensive monitoring and visualization capabilities. The project demonstrates practical implementation of Infrastructure as Code, CI/CD, container orchestration, and cloud-native monitoring techniques.